

Jour 2_complement : Protocole ARP et Gestion de Table (Article 1 - fin)

- **Pratique** : Décoder les entêtes Ethernet. Gestion des adresses MAC

Nous utilisons ce code qui va permettre de parser el contenu de la trame Ethernet et en utilisant la structure fourni dans le document sur [⊕ Let's code a TCP/IP stack, 1: Ethernet & ARP](#)
On peut obtenir un programme qui sépare les différents champs de la trame ethernet.

```
while (1) {
    int nread = read(tun_fd, buf, sizeof(buf));
    if (nread < 0) {
        perror("read");
        break;
    }
    if (nread < 14) continue; // Trop petit pour un header Ethernet

    struct eth_hdr *eth = (struct eth_hdr *)buf;

    // On convertit ethertype en little-endian pour l'afficher
    uint16_t type = ntohs(eth->ethertype);

    printf("\n=== Trame reçue (%d octets) ===\n", nread);
    printf("Dst MAC : %02x:%02x:%02x:%02x:%02x:%02x\n",
        eth->dmac[0], eth->dmac[1], eth->dmac[2],
        eth->dmac[3], eth->dmac[4], eth->dmac[5]);
    printf("Src MAC : %02x:%02x:%02x:%02x:%02x:%02x\n",
        eth->smac[0], eth->smac[1], eth->smac[2],
        eth->smac[3], eth->smac[4], eth->smac[5]);
    printf("EtherType : 0x%04x → ", type);

    switch (type) {
        case ETH_P_IP:    printf("IPv4\n");    break;
        case ETH_P_ARP:   printf("ARP\n");     break;
        case ETH_P_IPV6:  printf("IPv6\n");     break;
        default:          printf("Inconnu\n");  break;
    }
}
```

```
kali@kali: ~/jour2
Session Actions Edit View Help
root@kali: /home/kali/jour2 kali@kali: ~/jour2
(kali@kali)~/jour2
$ ping -I tap0 -c 2 10.0.0.2
PING 10.0.0.2 (10.0.0.2) from 10.0.0.1 tap0: 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1002ms
pipe 2
(kali@kali)~/jour2
$
```

```
root@kali: /home/kali/jour2
Session Actions Edit View Help
root@kali: /home/kali/jour2 kali@kali: ~/jour2
(root@kali)~/home/kali/jour2
$ ./opentap
5: tap0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP m
ode DEFAULT group default qlen 1000
    link/ether fa:6b:ba:7e:1b:d6 brd ff:ff:ff:ff:ff:ff

== Trame reçue (42 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : fa:6b:ba:7e:1b:d6
EtherType : 0x0806 → ARP

== Trame reçue (42 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : fa:6b:ba:7e:1b:d6
EtherType : 0x0806 → ARP

== Trame reçue (42 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : fa:6b:ba:7e:1b:d6
EtherType : 0x0806 → ARP
$
```

C'est mieux maintenant , on voit la trame de manière lisible et compréhensible au lieu d'une suite de bit sur wireshark.

Et on fait aussi l'implémentation du cache arp

```
root@kali: /home/kali/3_jour
Session Actions Edit View Help
root@kali: /home/kali/3_jour kali@kali: ~ root@kali: /home/kali
(root@kali)~/home/kali/3_jour
$ gcc *.c -o main
$ ./main
Interface tap0 ouverte (fd = 3) 1e:ef:02:90:ca:db Broadcast ARP 58 ARP
5: tap0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP m
ode DEFAULT group default qlen 1000
    link/ether 1e:ef:02:90:ca:db brd ff:ff:ff:ff:ff:ff

== Trame reçue (58 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : 1e:ef:02:90:ca:db
EtherType : 0x0806 → ARP

== Trame reçue (58 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : 1e:ef:02:90:ca:db
EtherType : 0x0806 → ARP

== Trame reçue (58 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : 1e:ef:02:90:ca:db
EtherType : 0x0806 → ARP
```

et vous verrez tout en bas que le **cache arp** est bien mis à jour

```
root@kali: /home/kali/3_jour
Session Actions Edit View Help
root@kali: /home/kali/3_jour kali@kali: ~ root@kali: /home/kali
Interface tap0 configurée sur 10.0.0.1. Votre programme doit utiliser 10.0.0.2

(root@kali)~/home/kali/3_jour
$ gcc *.c -o main

(root@kali)~/home/kali/3_jour
$ ./main
Interface tap0 ouverte (fd = 3)
7: tap0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 76:d3:e9:e3:8f:1e brd ff:ff:ff:ff:ff:ff

== Trame reçue (58 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : 76:d3:e9:e3:8f:1e
EtherType : 0x0806 → ARP

--- Cache ARP Interne ---
[0] 10.0.0.1 → 76:d3:e9:e3:8f:1e

ARP Reply envoyé à 10.0.0.1

== Trame reçue (58 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : 76:d3:e9:e3:8f:1e
EtherType : 0x0806 → ARP

--- Cache ARP Interne ---
[0] 10.0.0.1 → 76:d3:e9:e3:8f:1e 450

ARP Reply envoyé à 10.0.0.1

== Trame reçue (58 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : 76:d3:e9:e3:8f:1e
EtherType : 0x0806 → ARP

--- Cache ARP Interne ---
[0] 10.0.0.1 → 76:d3:e9:e3:8f:1e

ARP Reply envoyé à 10.0.0.1
```

Pour le code , il s'agit essentiellement de celui du dépôt git suivant :

saminiir/level-ip

A hacker's userspace TCP/IP stack



 2
Contributors

 15
Issues

 3k
Stars

 420
Forks



- **Focus Sécurité** : MAC Spoofing. Si je change un octet dans cet entête, je change mon identité physique sur le réseau.

Jour 3 : Datagrammes IPv4 (Article 2 - début)

- **Pratique : Parsing de l'entête IP, vérification du checksum et routage de base**

Donc on fait un ping sur 10.0.0.2 qui est l'adresse sur laquelle écoute le programme mais vous remarquerez que le ping prend un timeout. En fait c'est logique car pour l'instant, nous n'avons codé que :

- Lire la trame.
- Répondre à "Qui es-tu ?".
- Lire l'entête et vérifier le checksum.

Mais nous n'avons pas encore codé la réponse ICMP (tâche du jour 4).

